

DETC2026-194246

STRUCTURED REDUCED-BASIS NEURAL OPERATORS FOR PDES ON COMPLEX GEOMETRIES

Michael A.P. Warner¹, John T. Hwang¹,

¹University of California, San Diego, CA

ABSTRACT

Operator learning has emerged as a promising surrogate modeling approach for partial differential equations (PDEs), enabling rapid evaluation of physics-based models within outer-loop processes such as design optimization. However, many existing operator learning architectures assume simple domains or structured coordinate parameterizations, limiting their applicability to engineering problems defined on complex geometries. This paper introduces a structured reduced-basis neural operator architecture that incorporates an explicit high-dimensional discretization space between the input and output function spaces. The discretization space represents the geometry of the physical domain using standard numerical representations such as finite element meshes, while the neural network operates only on a low-dimensional latent representation defined within that space through compression and lifting maps whose complexity scales linearly with the discretization size. Multiple strategies for constructing the compression and lifting maps are considered, including fixed reduced bases obtained via Proper Orthogonal Decomposition (POD) as well as data-driven bases learned jointly with the operator model. This formulation separates geometric representation from operator approximation, allowing complex geometries to be represented without requiring the neural network to scale with the resolution of the underlying discretization. In addition, the resulting architecture is consistent with the discretization-invariant viewpoint commonly adopted in the neural operator literature. The proposed method is evaluated on a structural analysis problem for a blended wing-body aircraft model. Numerical experiments show that structured reduced-basis representations significantly improve both training behavior and predictive accuracy compared with a standard DeepONet architecture, highlighting the potential of this architecture for surrogate modeling of PDE-based engineering systems.

Keywords: Operator Learning, MDO

Distribution Statement A: Approved for Public Release; Distribution is Unlimited. PA# AFRL-2026-1143

1. INTRODUCTION

Computing the solutions to partial differential equations (PDEs) is central to many problems in engineering analysis and design. Particularly, multidisciplinary design optimization (MDO) frequently requires the repeated evaluation of PDE-based models in order to assess the effect of design parameters on physical fields such as structural displacements, stresses, or fluid flow. The field of MDO has been trending towards larger design spaces and greater model fidelity [1–3]. However, with these larger problems, the cost of repeatedly solving the governing PDEs can become a significant bottleneck.

Surrogate modeling techniques offer a promising approach to reducing this computational cost by learning approximate mappings between input parameters and solution fields [4, 5]. Recently, operator learning methods have emerged as a particularly attractive class of surrogates for PDE-based systems. Rather than approximating the solution at a fixed set of spatial locations, operator learning architectures learn mappings between function spaces, allowing the solution field to be evaluated at arbitrary spatial locations once the operator has been trained [6, 7]. Architectures such as Deep Operator Networks (DeepONets) [8] and neural operators such as the Fourier Neural Operator (FNO) [9] and Graph Neural Operator (GNO) [10] have demonstrated strong performance on a variety of benchmark PDE problems.

Despite this progress, most demonstrations of operator learning methods have focused on relatively simple domains and geometries [8, 11–13]. However, in many engineering applications, the underlying domain may be geometrically complex and, therefore, difficult to represent using the coordinate parameterizations typically assumed by neural operator architectures. For example, structural models of aircraft are often defined on irregular geometries represented by finite element meshes. At the same time, standard DeepONet formulations represent input functions through point samples at a fixed set of sensors, which ties the learned operator to a particular sampling pattern and does not satisfy the discretization-invariant formulations commonly used in the neural operator literature [14].

These considerations motivate the development of operator learning architectures that can both accommodate complex geometric representations and retain discretization-invariant properties. In this work, we introduce a structured reduced-basis neural operator architecture that explicitly separates the representation of domain geometry from the dimensionality of the learned operator approximation. The key idea is to introduce an intermediate high-dimensional discretization space that captures the structure of the physical domain, while the neural network operates only on a low-dimensional latent representation defined within that space. This formulation allows complex domains to be represented using standard discretization tools such as finite element meshes, while maintaining a compact neural operator model whose complexity is independent of the discretization resolution.

Within this framework, the operator learning process is factorized into a sequence of projection, compression, approximation, lifting, and reconstruction steps. The high-dimensional discretization space provides a flexible representation of the domain geometry, while the neural network learns a low-dimensional basis and operator mapping that act within this representation. As a result, the method combines the geometric flexibility of discretization-based representations with the parameter efficiency of low-rank operator learning architectures.

The main contribution of this work is the introduction of the aforementioned operator learning architecture that incorporates an explicit discretization space between the input and output function spaces. We show how this formulation separates geometric representation from operator approximation, demonstrate that the resulting architecture satisfies the discretization-invariant formulation commonly used in the neural operator literature, and evaluate the method on a structural analysis problem defined on a blended wing-body aircraft model, demonstrating improved accuracy and training behavior compared to a standard DeepONet architecture.

The remainder of this paper is organized as follows. Section 2 reviews parameterized PDE problems, operator learning architectures, and the concept of discretization invariance. Section 3 introduces the proposed operator architecture and discusses the selection of compression and lifting maps. Section 4 presents numerical experiments on an aircraft structural model. Finally, Section 5 summarizes the findings and outlines directions for future work.

2. BACKGROUND

2.1. Parameterized PDE Problems

Consider a parameterized partial differential equation defined over a domain Ω with parameters $\mu \in \mathcal{P}$,

$$\mathcal{N}(u(x; \mu), \mu) = 0, \quad x \in \Omega, \quad (1)$$

subject to appropriate boundary conditions. The solution $u(x; \mu)$ is a function defined over the domain Ω whose behavior depends on the parameter vector μ .

Many surrogate modeling approaches aim to approximate the mapping from parameters to the resulting solution field,

$$\mathcal{G} : \mathcal{P} \rightarrow V, \quad (2)$$

where V denotes a suitable function space defined over Ω .

More generally, operator learning methods consider mappings between function spaces,

$$\mathcal{G} : U \rightarrow V, \quad (3)$$

where U and V are spaces of input and output functions respectively. In the context of parameterized PDEs, the input function may represent parameter fields or forcing functions, while the output corresponds to the PDE solution [6].

The objective of operator learning methods is to construct an approximation $\hat{\mathcal{G}}$ that can efficiently evaluate solution fields for new inputs without directly solving the underlying PDE. Most operator learning architectures can be interpreted as consisting of three conceptual steps. First, the input function is *encoded* into a finite-dimensional representation. Second, the operator is *approximated* within this representation using a parametric model such as a neural network. Finally, the resulting representation is *decoded* to produce the output function in the target function space. This encoder–approximator–decoder decomposition provides a useful framework for comparing different operator learning architectures [15].

2.2. DeepONet

DeepONets approximate operators through a learned basis expansion. The architecture consists of two neural networks: a *branch network* and a *trunk network*.

The input function $u(x)$ is first encoded by sampling it at a fixed set of sensor locations $\{x_i\}_{i=1}^m$, producing the vector

$$u \rightarrow (u(x_1), \dots, u(x_m)). \quad (4)$$

This vector is provided as input to the branch network, which acts as the approximator. The branch network is a single fully connected neural network that maps the sensor values to a vector of coefficients,

$$\mathbf{c} = B_\theta(u) \in \mathbb{R}^P. \quad (5)$$

Here B_θ denotes the branch network parameterized by weights θ , and the entries of $\mathbf{c}$ correspond to the coefficients of a learned basis expansion.

The trunk network acts as the decoder. Given a spatial coordinate x , the trunk network evaluates a set of basis functions at that location,

$$\mathbf{t}(x) = T_\phi(x) \in \mathbb{R}^P, \quad (6)$$

where T_ϕ is a second fully connected neural network parameterized by weights ϕ . The entries of $\mathbf{t}(x)$ correspond to the values of p learned basis functions at the spatial location x . This standard DeepONet trunk is often modified, such as using Principal Component Analysis (PCA) to select the basis functions rather than learning them during training [11, 16].

The predicted output is obtained by combining the coefficients produced by the branch network with the basis functions produced by the trunk network,

$$\hat{\mathcal{G}}(u)(x) = \sum_{i=1}^p c_i t_i(x) = \mathbf{c}^\top \mathbf{t}(x). \quad (7)$$

Thus, the DeepONet represents the operator as a low-rank expansion in a learned basis defined by the trunk network, with coefficients determined by the branch network from the encoded input function.

2.3. Neural Operators

Neural operator architectures take an alternative approach for operator learning. Rather than representing the operator through sensors on the input function and a learned basis expansion for the output as in the DeepONet, neural operators typically approximate integral operators of the form

$$\mathcal{G}(u)(x) = \int_{\Omega} k(x, y) u(y) dy, \quad (8)$$

where $k(x, y)$ is a learned kernel [10, 14].

One widely used architecture in this class is the FNO [9]. The FNO can be interpreted within the encoder–approximator–decoder framework described above.

First, the input function $u(x)$ is encoded by projecting it onto a truncated Fourier basis. This produces a vector of Fourier coefficients

$$\hat{u}_k = \mathcal{F}(u)(k), \quad (9)$$

where $\mathcal{F}$ denotes the Fourier transform and k indexes the retained Fourier modes. These coefficients represent the input function in a spectral basis.

Next, the operator is approximated in this spectral space. Each neural operator layer applies a learned linear transformation to the retained Fourier coefficients,

$$\hat{v}_k = W_k \hat{u}_k, \quad (10)$$

where W_k are learned weight matrices. In practice, only a fixed number of low-frequency Fourier modes are retained, allowing the network to operate within a reduced spectral representation.

Finally, the output function is decoded by applying the inverse Fourier transform,

$$v(x) = \mathcal{F}^{-1}(\hat{v})(x), \quad (11)$$

which reconstructs the spatial field from the transformed Fourier coefficients.

Thus, similar to the DeepONet, the FNO ultimately represents the operator through a basis expansion,

$$\hat{\mathcal{G}}(u)(x) = \sum_k \hat{v}_k e^{ikx}, \quad (12)$$

where the Fourier modes e^{ikx} act as fixed basis functions and the neural network determines the corresponding coefficients. Because the Fourier basis is defined independently of any particular spatial discretization, the learned operator can be applied to different discretizations of the domain.

2.4. Discretization Invariance

A key concept in the neural operator literature is discretization invariance. Following the definition by Kovachki et al. [14], this property does not require the absence of a discretization. Rather, it requires that a model define a continuous operator whose discrete implementations converge as the input discretization is refined. More precisely, given a parametric class of operators

$$G : A \times \Theta \rightarrow U,$$

a model is discretization-invariant if, for each fixed parameter vector $\theta \in \Theta$, there exists a sequence of discrete maps $\hat{G}_L$ such that the discretized uniform risk tends to zero as the discretization D_L is refined. The discretized uniform risk is defined as

$$R_K(G, \hat{G}, D_L) = \sup_{a \in K} \|\hat{G}(D_L, a|_{D_L}) - G(a)\|_U,$$

for any compact set $K \subset A$. Thus, discretization invariance is fundamentally a consistency property: as finer information about the input function becomes available, the discrete evaluation of the model must converge to the action of an underlying continuum operator [14].

Under this definition, neural operator architectures such as the FNO are discretization-invariant because they define an operator in function space and use the spatial discretization only to numerically evaluate that operator. In the FNO, the learned parameters act on Fourier coefficients, while refinement of the spatial grid only improves the numerical approximation of those coefficients. Consequently, for a fixed set of learned parameters, the discrete implementations converge to the same continuum operator as the input discretization is refined. This is the sense in which the FNO is discretization-invariant; it does not imply that the model is free of basis truncation error or independent of the chosen spectral representation.

The DeepONet behaves differently. On the output side, the trunk network defines basis functions that can be evaluated at arbitrary spatial locations, so the predicted output can be queried independently of the discretization used for training. On the input side, however, the branch network receives point evaluations of the input function at a fixed set of sensor locations. This ties the representation of the input to a particular sampling pattern, and therefore the resulting architecture does not satisfy discretization invariance.

An important note is that this definition is narrower than the phrase "discretization invariance" may initially suggest. It does not require that a method avoid discretizations altogether, nor does it require that the model be insensitive to the choice of representation space. Rather, it requires that the learned parameters define a continuum operator whose discrete evaluations are consistent under spatial refinement. This distinction is useful in interpreting the proposed architecture.

3. STRUCTURED REDUCED-BASIS NEURAL OPERATOR

3.1. Proposed Architecture

The proposed architecture is motivated by two objectives. First, we seek an operator learning formulation that can naturally accommodate complex geometry, which can be challenging

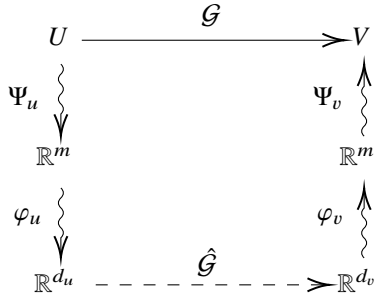


FIGURE 1: Two-step encoding and decoding architecture with intermediate high-dimensional discretization space

for neural operator architectures that rely on simple underlying domains or structured coordinate parameterizations. Second, we seek a formulation that satisfies the discretization-invariance viewpoint discussed in the previous section, in contrast to a standard DeepONet whose input representation is tied to a fixed set of sensor locations. To achieve these goals, the proposed architecture introduces an explicit intermediate discretization space that serves as a representation of the domain or geometry while allowing the learned operator itself to act in a low-dimensional latent space. The resulting process is illustrated in Fig. 1.

Given an input function $u \in U$, the encoder first projects it into a fixed high-dimensional space $\mathbb{R}^m$ associated with a chosen discretization using the projection operator

$$\Psi_u : U \rightarrow \mathbb{R}^m.$$

If $\mathbb{R}^m$ is associated with a finite-dimensional approximation space spanned by basis functions $\{\psi_i\}_{i=1}^m$, then this projection may be interpreted as finding coefficients $d \in \mathbb{R}^m$ such that

$$u_h(x) = \sum_{i=1}^m d_i \psi_i(x),$$

with the coefficients determined, for example, through a Galerkin projection

$$\int_{\Omega} \psi_j(x) u_h(x) dx = \int_{\Omega} \psi_j(x) u(x) dx, \quad j = 1, \dots, m.$$

Equivalently,

$$\sum_{i=1}^m \left(\int_{\Omega} \psi_j(x) \psi_i(x) dx \right) d_i = \int_{\Omega} \psi_j(x) u(x) dx, \quad j = 1, \dots, m.$$

In practice, these integrals are evaluated numerically using an appropriate quadrature rule, so the discrete realization of Ψ_u depends on the chosen quadrature scheme while converging to the same continuum projection as the numerical approximation is refined.

This high-dimensional representation is then compressed to a low-dimensional latent vector by

$$\varphi_u : \mathbb{R}^m \rightarrow \mathbb{R}^{d_u},$$

producing

$$z = \varphi_u(\Psi_u(u)) \in \mathbb{R}^{d_u}.$$

The approximation step is performed in this latent space. A branch network

$$\hat{\mathcal{G}} : \mathbb{R}^{d_u} \rightarrow \mathbb{R}^{d_v}$$

maps the encoded input to a coefficient vector

$$c = \hat{\mathcal{G}}(z) \in \mathbb{R}^{d_v}.$$

Decoding proceeds in two stages. First, the coefficients are lifted back to a high-dimensional discretization space through the map

$$\varphi_v : \mathbb{R}^{d_v} \rightarrow \mathbb{R}^m.$$

In practice, this lifting is implemented as a learned linear basis,

$$\varphi_v(c) = \Phi c, \quad \Phi \in \mathbb{R}^{m \times d_v},$$

where the columns of Φ represent learned basis functions defined over the chosen discretization. This produces a high-dimensional field representation on the output discretization.

Second, a reconstruction operator

$$\Psi_v : \mathbb{R}^m \rightarrow V$$

maps this discretized field back to the continuous output space, allowing the predicted field to be evaluated at arbitrary spatial locations using the reconstruction associated with the chosen discretization (e.g., interpolation through finite element shape functions).

The resulting operator approximation is therefore

$$\hat{\mathcal{G}} = \Psi_v \circ \varphi_v \circ \hat{\mathcal{G}} \circ \varphi_u \circ \Psi_u. \quad (13)$$

For notational simplicity, the same intermediate dimension m has been used for both the input and output discretization spaces. In practice, however, these spaces need not be the same. The input and output may be represented on entirely different discretizations, with different dimensions, basis functions, and even different underlying domains. This is particularly important in multiphysics or transfer settings, where the input and output quantities may naturally live on different geometric representations. The formulation above is unchanged in spirit if one instead writes

$$\Psi_u : U \rightarrow \mathbb{R}^{m_u}, \quad \Psi_v : \mathbb{R}^{m_v} \rightarrow V,$$

with corresponding maps

$$\varphi_u : \mathbb{R}^{m_u} \rightarrow \mathbb{R}^{d_u}, \quad \varphi_v : \mathbb{R}^{d_v} \rightarrow \mathbb{R}^{m_v}.$$

This construction separates geometric representation from approximation complexity. The high-dimensional discretization spaces are responsible for representing the structure of the underlying input and output domains, while the learned operator acts only on the low-dimensional latent spaces $\mathbb{R}^{d_u}$ and $\mathbb{R}^{d_v}$. As a result, the expressive power of the discretization can be increased without requiring the approximation network itself to scale with the number of spatial degrees of freedom.

3.2. Selection of the Compression and Lifting Maps

A unique design choice in the proposed architecture is the selection of the compression and lifting maps,

$$\varphi_u : \mathbb{R}^{m_u} \rightarrow \mathbb{R}^{d_u}, \quad \varphi_v : \mathbb{R}^{d_v} \rightarrow \mathbb{R}^{m_v}.$$

These maps determine how the high-dimensional discretization spaces are related to the low-dimensional latent spaces in which the operator approximation is performed. There are two fundamental ways of selecting these maps: they may be specified *a priori* and held fixed during training, or they may be learned jointly with the approximation network (or one may be fixed while the other is learned). The choice introduces tradeoff between inductive bias, flexibility, and training complexity.

When prescribing these maps directly, the training reduces to learning the map between two preselected reduced spaces. This gives considerable freedom in choosing the representation. A familiar example to the world of reduced order modeling would be to choose the compression and lifting maps from a proper orthogonal decomposition (POD) basis computed from training snapshots. Given a set of discretized fields

$$\{y^{(1)}, y^{(2)}, \dots, y^{(N)}\} \subset \mathbb{R}^m,$$

POD constructs an orthonormal basis that optimally captures the energy of these "snapshots" in a least-squares sense [17]. If the snapshot matrix is written as

$$Y = \begin{bmatrix} y^{(1)} & y^{(2)} & \dots & y^{(N)} \end{bmatrix},$$

then the POD basis is obtained from the leading singular vectors of Y or, equivalently, from the dominant eigenvectors of the associated correlation matrix. Retaining the first r modes yields a basis

$$\Phi_r \in \mathbb{R}^{m \times r},$$

and the associated compression and lifting operators may be chosen as

$$\varphi(y) = \Phi_r^\top y, \quad \varphi^{-1}(c) = \Phi_r c.$$

This is attractive when the dominant variability of the fields is well captured by a low-dimensional linear subspace.

Another possibility for a prescribed map is to use a spectral transform. For problems defined on domains compatible with such a representation, one may choose Fourier modes or other global basis functions, as the compression and lifting maps. In this case, the architecture becomes conceptually similar to the Fourier Neural Operator. More generally, any prescribed basis for the discretization space can be used in this way, including wavelet, polynomial, or radial basis expansions.

A third option is to use the identity map. If the intermediate discretization dimension m is already sufficiently small, then one may simply take

$$\varphi_u = I, \quad \varphi_v = I,$$

so that the neural network operates directly in the discretization space without any additional dimensionality reduction. This corresponds to the special case $d_u = m_u$ and $d_v = m_v$. While this sacrifices the parameter savings associated with a reduced latent space, it may be appropriate when the discretization itself

is already modest in size or when preserving the full discretized representation is desirable.

Learning these maps from the data results in an architecture more similar to the DeepONet. Specifically, this corresponds to learning the lifting map φ_v jointly with the approximation network, so that the basis used to reconstruct the high-dimensional field adapts to the operator learning task. Similarly, the input compression map φ_u may be learned, allowing the model to discover an input representation that is most useful for prediction. One may also adopt a hybrid strategy, for example by fixing φ_u using a POD or Fourier basis while learning φ_v , or vice versa. Such hybrid choices allow prior knowledge to be incorporated on one side of the operator while preserving flexibility on the other.

These alternatives provide a common framework for understanding several existing approaches. Fixed POD-based compression and lifting recover a reduced-basis operator model. Fixed Fourier transforms lead to a spectral representation reminiscent of the FNO. Learning the lifting map while keeping the input approximation low-dimensional yields the structured basis formulation emphasized in this work. The proposed architecture is therefore not tied to a single choice of reduced representation. Instead, it provides a general framework in which the discretization space is separated from the latent operator approximation, and the compression and lifting maps determine how this separation is realized in practice.

3.3. Revisiting Discretization Invariance

As previously discussed, the distinction between discretization-dependent and discretization-invariant models is more subtle than a simple dichotomy between using and not using a discretization. In fact, neural operators represent functions using finite-dimensional spaces that effectively discretize the domain. The difference to the proposed architecture is that most use global basis functions, such as Fourier modes, rather than local basis functions as in a mesh. However, the use of such a representation does not prevent a method from satisfying discretization-invariance.

In the present work, the maps

$$\Psi_u : U \rightarrow \mathbb{R}^m, \quad \Psi_v : \mathbb{R}^m \rightarrow V$$

define an explicit intermediate discretization space, while

$$\varphi_u : \mathbb{R}^m \rightarrow \mathbb{R}^{d_u}, \quad \hat{\mathcal{G}} : \mathbb{R}^{d_u} \rightarrow \mathbb{R}^{d_v}, \quad \varphi_v : \mathbb{R}^{d_v} \rightarrow \mathbb{R}^m$$

define the learned low-dimensional representation and operator approximation within that space. If the projection Ψ_u is implemented so that its numerical approximation converges under refinement of the discretization used to compute it, then the overall architecture defines a continuum operator together with a sequence of discrete approximations. In this sense, the proposed method satisfies the same notion of discretization invariance: refinement affects only the numerical realization of the projections, not the learned operator itself.

A useful clarifying example is a mesh-to-mesh network, which is the case when the compression and lifting maps are identity. Suppose a function is first projected onto a fixed mesh basis, then mapped by a neural network from coefficients on

that mesh to coefficients on another field defined over the same mesh, and finally reconstructed as a function. Such a method clearly depends on a chosen discretization space as a representation. Nevertheless, if the projection from the continuous input function into that mesh basis is computed more accurately as the input discretization is refined (e.g., using a higher-order quadrature scheme), then the corresponding discretized implementations converge to an underlying continuum operator. Thus, such a method is discretization-invariant.

This observation motivates the present formulation. Rather than avoiding discretization, the proposed architecture uses a high-dimensional representation space because such spaces are well suited to capturing complex geometry. A mesh or other rich discretization can accurately represent irregular domains, boundary conditions, and localized structure. At the same time, the neural network itself need not operate directly in this high-dimensional space. By introducing a learned low-dimensional representation on top of the discretization, the approximation is performed in a compact latent space whose dimension is independent of the resolution of the underlying representation. The result is an architecture that combines the geometric fidelity of high-dimensional discretizations with the parameter efficiency of low-dimensional operator representations. This also allows for a high degree of flexibility in the selection of the latent spaces, which can allow for better approximation performance.

4. NUMERICAL EXPERIMENTS

4.1. Problem Setup

This section compares several variants of the proposed structured-basis operator architecture with a standard DeepONet on a partial differential equation (PDE) problem defined on a realistic engineering geometry. The goal of this experiment is to evaluate the effect of different basis initialization and training strategies, as well as to compare the proposed architecture with a conventional DeepONet formulation.

The PDE problem considered in this study is based on a Reissner-Mindlin shell formulation defined over the structure of a blended wing-body (BWB) aircraft. An overview of shell methods can be found in [18]. The structure is subjected to a fixed aerodynamic loading condition while the structural response varies with the distribution of material thickness. The thickness field is parameterized using B-splines defined across the surface of the structure, with a total of 26 control points serving as the input parameters to the operator network. A total of 5,000 samples were generated, with 4,000 used for training and 1,000 for validation.

Because the thickness discretization is already low-dimensional, it was used directly as the input space. The fixed output discretization used by the structured-basis architecture consists of a linear triangular mesh containing 3,964 nodes. Over this discretization, a total of 192 basis functions were defined, corresponding to 64 basis functions for each component of the three-dimensional displacement vector.

Three variants of the structured-basis architecture were evaluated. In the first case, *learned (rand)*, the basis functions were initialized with random Gaussian noise and subsequently optimized during network training. In the second case, *learned (POD)*, the basis was initialized using POD modes computed

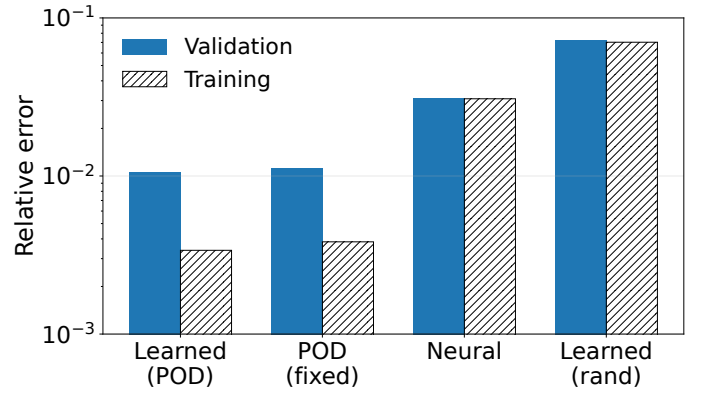


FIGURE 2: Training and validation errors for different trunk architectures.

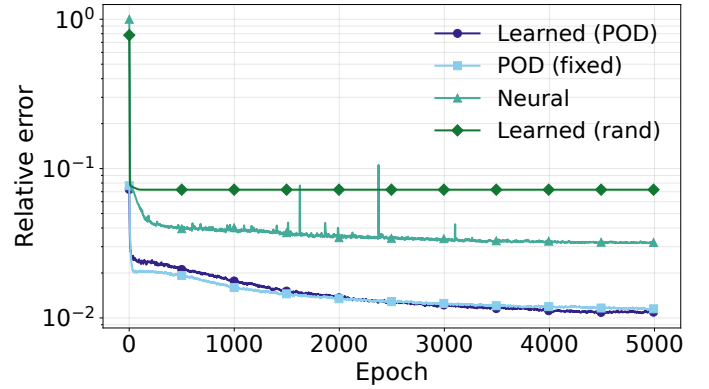


FIGURE 3: Validation error during training for architectures.

from the training data and was allowed to further adapt during training. In the third case *POD (fixed)*, the basis was also initialized using POD modes but remained fixed during training, corresponding to a POD-based reduced representation.

For comparison, a fourth case, *neural*, consisting of a standard DeepONet architecture was also evaluated. All four cases used the same branch network configuration in order to isolate the effect of the trunk representation. The trunk network in the DeepONet case was chosen such that the total number of parameters approximately matched that of the structured-basis trunk. The resulting architectural configurations are summarized in Table 1.

4.2. Results

Figure 2 shows the training and validation losses for the four network configurations.

Among the tested models, the structured-basis network initialized with POD modes and subsequently learned during training achieved the lowest training and validation error. The configuration using a fixed POD basis performed nearly as well, with only a small gap in the final validation loss.

This behavior is consistent with the properties of POD bases. POD provides the optimal linear basis for reconstructing the training snapshots with minimal mean-square error, resulting in a strong representation for the displacement fields in the dataset even with a fixed basis. However, the optimal basis for recon-

Network Component	Input Dim.	Hidden Layers	Output Dim.	# Parameters
Branch Network	26	384×3	192	391,860
Neural Trunk	3	576×3	192	777,795
Reduced-Basis Trunk	3964	—	192	761,088

TABLE 1: Network architecture and parameter counts for the branch network and trunk variants used in the numerical experiments. The branch network is shared across all cases, while the trunk is either implemented as a neural network or as a learned reduced basis defined on the discretization.

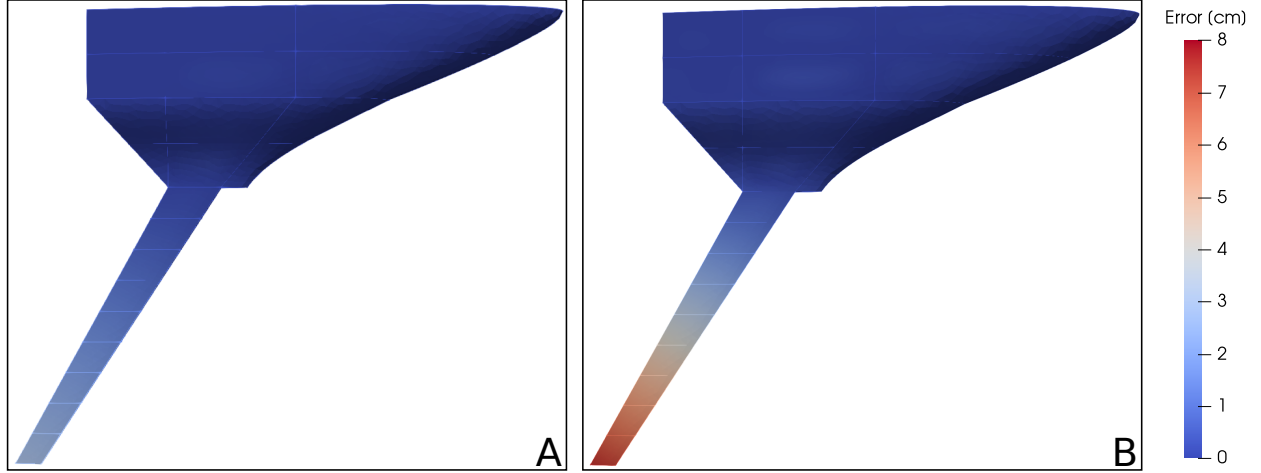


FIGURE 4: Absolute error in predicted displacement for neural network with (A) learned mesh basis and (B) neural basis.

structing snapshots is not necessarily the optimal basis for representing the operator mapping, especially combined with the dynamics of training the branch network. Allowing the basis functions to adapt during training therefore provides additional expressivity, which can reduce the final approximation error. In the present case this improvement is modest but consistent.

The neural trunk network and the randomly initialized learned basis both produced significantly higher training and validation errors. In both cases the model must simultaneously learn a suitable basis representation and the operator mapping itself, which appears to make optimization more difficult. As a result, these architectures converge to higher-error solutions.

Figure 3 shows the relative validation error as a function of training iteration.

The models initialized with POD bases exhibit a rapid initial decrease in validation error, followed by a slower improvement phase as training progresses. This behavior reflects the strong initial representation provided by the POD basis: even without further adaptation, the basis already captures the dominant spatial structures present in the displacement fields.

Allowing the basis to update during training yields a small but measurable improvement over the fixed POD case. Most of this improvement occurs late in training, suggesting that the network first learns the operator mapping within the POD subspace and subsequently refines the basis to better align with the operator learning task. This observation suggests a potential training strategy in which the basis is initially fixed while the branch network is trained, and only later allowed to adapt once the operator mapping has largely converged.

In contrast, the neural trunk and randomly initialized ba-

sis both plateau at relatively large errors. These architectures must discover an appropriate spatial representation while simultaneously learning the operator mapping, which appears to slow convergence and limit the final accuracy.

To visualize the effect of these architectural differences, Fig. 4 shows the spatial distribution of displacement prediction error for a randomly selected validation case. The comparison focuses on two representative models: the neural trunk DeepONet and the structured-basis model initialized with POD and learned during training.

In both cases, the largest prediction errors occur near the wing tip, where the structural displacements are largest. However, the magnitude of the error differs substantially between the two models, with the POD-initialized structured-basis network produces errors significantly below that of the neural trunk network across the structure.

More subtle differences are also visible in the body of the aircraft. The neural trunk model shows noticeable error patterns in the skin panels between ribs and spars, corresponding to local bending deformation of the shell structure. These errors are nearly absent in the POD-initialized structured-basis model, indicating that the learned basis provides a more accurate representation of the relevant structural deformation modes.

Overall, these results suggest that incorporating a structured discretization space together with a low-dimensional learned basis significantly improves both the accuracy and stability of operator learning for problems with complex geometry.

5. CONCLUSION

This paper introduced a structured operator learning architecture that incorporates an explicit discretization space between

the input and output function spaces. By separating geometric representation from operator approximation, the method enables neural operator models to be applied to complex engineering geometries represented using standard discretizations such as finite element meshes. Numerical experiments on a blended wing-body structural model demonstrated that structured reduced-basis representations significantly improved predictive accuracy compared with a conventional DeepONet architecture. These results suggest that combining high-dimensional discretizations with low-dimensional operator representations is a promising direction for operator learning in engineering analysis and design.

ACKNOWLEDGMENTS

The authors would like to acknowledge the Multidisciplinary Science and Technology Center of the Aerospace Systems Directorate, Air Force Research Laboratory for funding and supporting this effort.

REFERENCES

- [1] Sobieszcanski-Sobieski, J. and Haftka, R. T. “Multidisciplinary aerospace design optimization: survey of recent developments.” *Structural optimization* Vol. 14 No. 1 (1997): pp. 1–23. DOI [10.1007/BF01197554](https://doi.org/10.1007/BF01197554).
- [2] Martins, Joaquim R.R.A. and Lambe, Andrew B. “Multidisciplinary design optimization: A survey of architectures.” *AIAA Journal* Vol. 51 (2013): pp. 2049–2075. URL <https://api.semanticscholar.org/CorpusID:18609991>.
- [3] Ruh, Marius L., Fletcher, Andrew, Sarojini, Darshan, Sperry, Mark, Yan, Jiayao, Scotzniovsky, Luca, van Schie, Sebastiaan P., Warner, Michael, Orndorff, Nicholas C., Xiang, Ru, Joshy, Anugrah J., Zhao, Han, Krokowski, Joshua, Gill, HyunJune, Lee, Seongkyu, Cheng, Zeyu, Cao, Zhi, Mi, Chris, Silva, Christopher, Wolfe, Lauren, Chen, Jiun-Shyan and Hwang, John T. *Large-scale multidisciplinary design optimization of a NASA air taxi concept using a comprehensive physics-based system model* (2024): DOI [10.2514/6.2024-0771](https://doi.org/10.2514/6.2024-0771).
- [4] Shi, Renhe, Long, Teng, Ye, Nianhui, Wu, Yufei, Wei, Zhao and Liu, Zhenyu. “Metamodel-based multidisciplinary design optimization methods for aerospace system.” *Astrodynamics* Vol. 5 No. 3 (2021): pp. 185–215. DOI [10.1007/s42064-021-0109-x](https://doi.org/10.1007/s42064-021-0109-x).
- [5] Forrester, Alexander, Sobester, Andras and Keane, Andy. *Engineering Design Via Surrogate Modelling: A Practical Guide* (2008). DOI [10.1002/9780470770801](https://doi.org/10.1002/9780470770801).
- [6] Chen, Tianping and Chen, Hong. “Universal approximation to nonlinear operators by neural networks with arbitrary activation functions and its application to dynamical systems.” *IEEE transactions on neural networks* Vol. 6 No. 4 (1995): pp. 911–917.
- [7] Boullé, Nicolas and Townsend, Alex. “A mathematical guide to operator learning.” Mishra, Siddhartha and Townsend, Alex (eds.). *Numerical Analysis Meets Machine Learning*. Vol. 25 of *Handbook of Numerical Analysis*. Elsevier (2024): pp. 83–125. DOI [10.1016/bs.hna.2024.05.003](https://doi.org/10.1016/bs.hna.2024.05.003).
- [8] Lu, Lu, Jin, Pengzhan, Pang, Guofei, Zhang, Zhongqiang and Karniadakis, George Em. “Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators.” *Nature Machine Intelligence* Vol. 3 No. 3 (2021): pp. 218–229. DOI [10.1038/s42256-021-00302-5](https://doi.org/10.1038/s42256-021-00302-5).
- [9] Li, Zongyi, Kovachki, Nikola B., Azizzadenesheli, Kamyar, Liu, Burigede, Bhattacharya, Kaushik, Stuart, Andrew M. and Anandkumar, Anima. “Fourier Neural Operator for Parametric Partial Differential Equations.” *CoRR* Vol. abs/2010.08895 (2020). URL <https://arxiv.org/abs/2010.08895>.
- [10] Li, Zongyi, Kovachki, Nikola, Azizzadenesheli, Kamyar, Liu, Burigede, Bhattacharya, Kaushik, Stuart, Andrew and Anandkumar, Anima. “Neural Operator: Graph Kernel Network for Partial Differential Equations.” (2020). URL <https://arxiv.org/abs/2003.03485>.
- [11] Lu, Lu, Meng, Xuhui, Cai, Shengze, Mao, Zhiping, Goswami, Somdatta, Zhang, Zhongqiang and Karniadakis, George Em. “A comprehensive and fair comparison of two neural operators (with practical extensions) based on FAIR data.” *Computer Methods in Applied Mechanics and Engineering* Vol. 393 (2022): p. 114778. DOI [10.1016/j.cma.2022.114778](https://doi.org/10.1016/j.cma.2022.114778).
- [12] Zhong, Weiheng, Liu, Qibang, Abueidda, Diab, Koric, Seid and Meidani, Hadi. “A comprehensive comparison of neural operators for 3D industry-scale engineering designs.” (2025). URL <https://arxiv.org/abs/2510.05995>.
- [13] Bhattacharya, Kaushik, Hosseini, Bamdad, Kovachki, Nikola B. and Stuart, Andrew M. “Model Reduction And Neural Networks For Parametric PDEs.” *The SMAI Journal of computational mathematics* Vol. 7 (2021): pp. 121–157. DOI [10.5802/smai-jcm.74](https://doi.org/10.5802/smai-jcm.74).
- [14] Kovachki, Nikola, Li, Zongyi, Liu, Burigede, Azizzadenesheli, Kamyar, Bhattacharya, Kaushik, Stuart, Andrew and Anandkumar, Anima. “Neural Operator: Learning Maps Between Function Spaces With Applications to PDEs.” *Journal of Machine Learning Research* Vol. 24 No. 89 (2023): pp. 1–97. URL <http://jmlr.org/papers/v24/21-1524.html>.
- [15] Lanthaler, Samuel, Mishra, Siddhartha and Karniadakis, George E. “Error estimates for DeepONets: a deep learning framework in infinite dimensions.” *Transactions of Mathematics and Its Applications* Vol. 6 No. 1 (2022): p. tnac001. DOI [10.1093/imatrm/tnac001](https://doi.org/10.1093/imatrm/tnac001).
- [16] Eivazi, Hamidreza, Wittek, Stefan and Rausch, Andreas. “Nonlinear model reduction for operator learning.” (2024). URL <https://arxiv.org/abs/2403.18735>.
- [17] Sirovich, Lawrence. “Turbulence and the dynamics of coherent structures. I. Coherent structures.” *Quarterly of Applied Mathematics* Vol. 45 (1987): pp. 561–571. DOI [10.1090/qam/910462](https://doi.org/10.1090/qam/910462).
- [18] Bischoff, M., Bletzinger, K.-U., Wall, W. A. and Ramm, E. *Models and Finite Elements for Thin-Walled Structures*. John Wiley Sons, Ltd (2004): Chap. 3. DOI [10.1002/0470091355.ecm026](https://doi.org/10.1002/0470091355.ecm026).